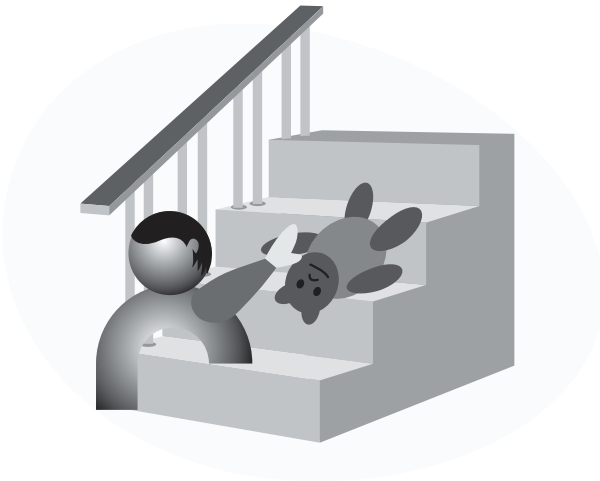


FIGURE 22.1 Edward Bear illustrating the need for a retrospective

every sprint (see Figure 22.2), allowing teams to take advantage of insights and data before they are lost.

Because a Scrum team meets at the end of each sprint to inspect and adapt its Scrum process, it can apply early and incremental learning throughout the development process and thereby significantly affect the outcome of the project.

In the rest of this chapter I describe a detailed approach for performing sprint retrospectives. However, don't let the details mislead you into believing that a sprint retrospective is a heavyweight, ceremonial process. A sprint retrospective can be as simple as the Scrum team members coming together to discuss questions such as

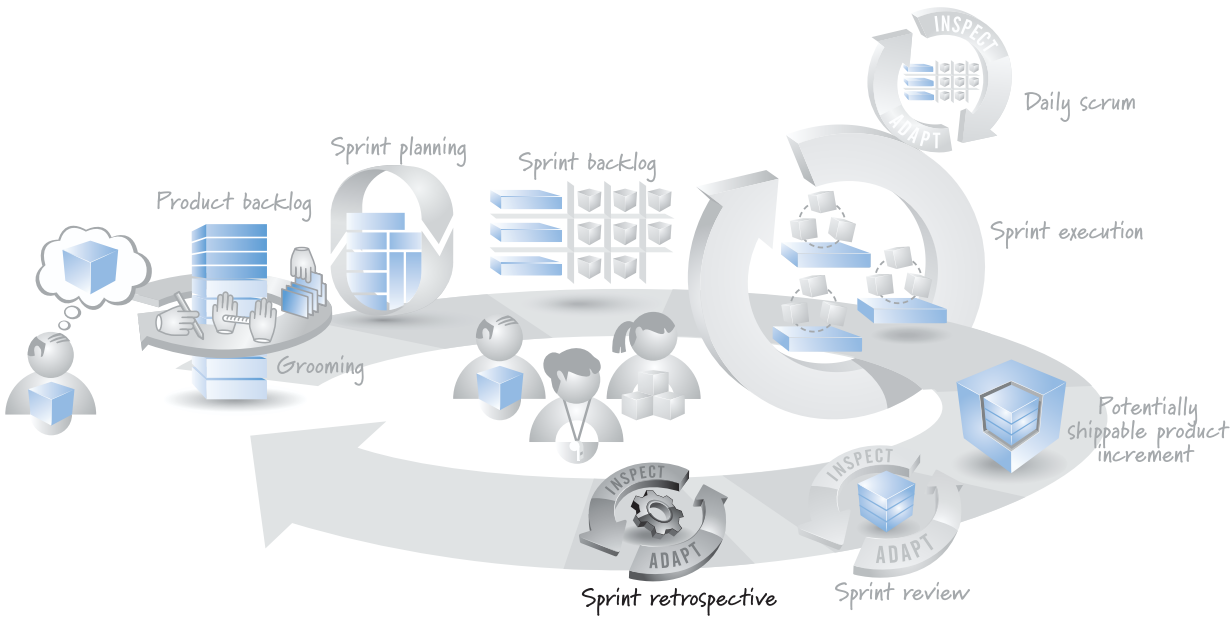


FIGURE 22.2 When the sprint retrospective happens

- What worked well this sprint that we want to continue doing?
- What didn't work well this sprint that we should stop doing?
- What should we start doing or improve?

Based on their discussions, team members determine a few actionable changes to make and then get on with the next sprint with an incrementally improved process.

Participants

Because the sprint retrospective is a time to reflect on the process, we need the full Scrum team to attend. This includes all members of the development team, the ScrumMaster, and the product owner. The development team includes everyone who is designing, building, and testing the product. Collectively, these team members have a rich and diverse set of perspectives that are essential for identifying process improvements from multiple points of view.

The ScrumMaster attends, both because she is an integral part of the process and also because she is the process authority for the Scrum team (see Chapter 10). Being an authority doesn't mean that the ScrumMaster should tell the team how to change its process. Instead, it means that she can point out where the team is not adhering to its own agreed-upon process and also be a valuable source of knowledge and ideas for the team.

Some argue that having the product owner at the retrospective might inhibit the team from being completely honest or revealing difficult issues. While this can be a risk in some organizations, the product owner is a critical part of the Scrum process and as such should be part of discussions about that process. If there is a lack of trust between the product owner and the development team, or there is a low level of safety so that speaking candidly isn't comfortable, perhaps the product owner should not attend until the ScrumMaster can help coach those involved toward creating a safer, more trusting environment.

Assuming trust and safety are reasonably in place, an effective product owner is critical to achieving the fast, flexible flow of business value and therefore should participate in the sprint retrospective. For example, the product owner is the channel or conduit through which requirements flow to the team. What if something is wrong with how requirements are flowing through the Scrum process? Perhaps PBIs are not well groomed by the start of sprint planning. In such cases it would be difficult for the Scrum team to brainstorm potential process improvements if the product owner were absent from the retrospective.

Stakeholders or managers who are not on a Scrum team, on the other hand, should attend a retrospective only if invited by the Scrum team. Although transparency is a core Scrum value, the reality is that many organizations have not yet achieved a level of safety to support non-Scrum team members regularly attending retrospectives. The team members must feel safe if they are to have an open and

candid discussion without feeling inhibited by outsiders. If the team doesn't feel safe enough to reveal the real issues because outsiders are attending, the retrospective will lose its effectiveness.

Prework

Prior to the sprint retrospective there is some prework to complete (see Figure 22.3).

For short-duration sprints or for teams that are using a well-practiced, simple retrospective format, this prework should not require much, if any, time.

Define the Retrospective Focus

Each sprint retrospective should have a well-defined focus. The default focus is to review all relevant aspects of the process the Scrum team used during the current sprint. However, there are times when a team might select a different retrospective focus based on what is currently important to the team and where it is energetic about seeing improvement. For example:

- Focus on how to improve our skills with test-driven development (TDD).
- Focus on why we build what we think the customers want, but when they see it they frequently believe we misunderstood their desires or missed an important facet of the requirement.

Establishing and communicating the focus before the start of the retrospective allow the Scrum team to determine if any non-Scrum team members should be invited. In addition, knowing the focus before the start of the retrospective allows the team to select appropriate retrospective exercises and gives people time to gather and prepare any data needed to ensure a smooth performance of the retrospective.

Having the ability to define a specific focus can help long-lived, high-performance Scrum teams continue to extract measurable value from sprint retrospectives. For example, in one organization I coached there was a mature Scrum team whose

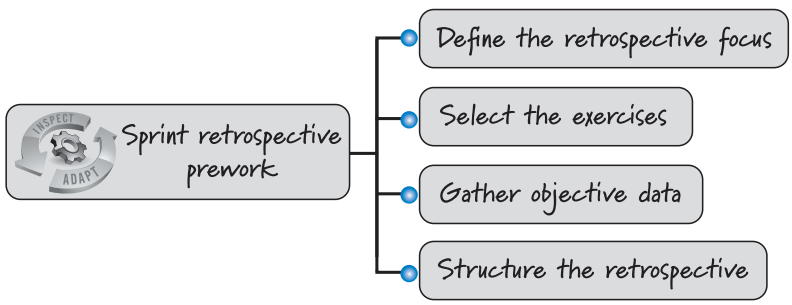


FIGURE 22.3 Sprint retrospective prework

members had been working well together for nearly three years. They had gone through many dozens of sprints together. They were starting to feel that doing a sprint retrospective focused on the just-completed sprint was often low value. One team member remarked, "For a long time the sprint retrospectives felt indispensable, and now they frequently just feel like process for the sake of process." What we ended up doing was performing shorter, more focused sprint retrospectives that allowed the team and invited outsiders to dig into very specific issues, going deep into root cause analysis. The result was that the team continued to learn and improve despite its considerable experience with Scrum. There is always room for growth; it just might require a more focused retrospective to uncover it.

Select the Exercises

Once we have established the focus and final participants for the upcoming retrospective, we can determine which exercises might help participants to engage, think, explore, and decide together. A typical retrospective includes the following exercises:

- Create and mine a sprint event timeline.
- Brainstorm insights.
- Group and vote on insights.

However, we might choose to vary these exercises to support a particular focus or set of participants. We might also decide to try new exercises to keep things fresh. See *Project Retrospectives* (Kerth 2001) and *Agile Retrospectives* (Derby and Larsen 2006) for additional exercises.

The participants don't have to decide exactly which exercises to use during prework. In fact, it might actually be better to select some exercises in a just-in-time fashion during the retrospective based on what the participants think would work best. At the same time, some exercises, especially those that require data or supplies, are best determined during the prework. Be prepared but stay flexible.

Gather Objective Data

Because a sprint retrospective is performed in a focused, short period of time (many teams establish a timebox), any legwork to collect needed data should be done before the retrospective begins.

We know both the focus and the exercise options for the upcoming retrospective, so we should have a good idea of what, if any, objective data should be gathered. Objective data is hard data (not opinions), such as what events happened and when, or counts of the number of PBIs that were started but not finished, or the feature burnup chart for the sprint illustrating the flow of completed work. At this point we are not organizing or analyzing any data; we are just collecting it so that it is available during the retrospective.

Structure the Retrospective

Like sprint reviews, retrospectives happen at the end of each sprint, often immediately following the sprint review, and generally should recur at the same place, day, and time each sprint. However, unlike for sprint reviews, you might occasionally need to vary the place, date, or time of a particular retrospective to better serve its focus, any non–Scrum team participants, or specific exercises you are planning to run. That's why I like to review the structure of the retrospective as part of the prework.

The exact length of the retrospective is influenced by factors such as how many people are on the team, how new the team is, whether any team members are located remotely, and so on. In my experience, teams new to Scrum have a tendency to budget too little time for their retrospectives. It's difficult to hold a meaningful sprint retrospective in less than 60 minutes. As a rule, I usually budget about 1.5 hours for the sprint retrospective when using two-week sprints, and proportionally more when using longer sprints.

The Scrum team should choose a sprint retrospective location that is most conducive to achieving a successful outcome. Some teams prefer to hold their retrospectives in the standard team area where their big visible charts are located. This gives them easy access to a wealth of relevant information. Others prefer to meet away from the standard team area, perhaps to introduce an environment with less emotional saturation where people might feel less inhibited and more likely to speak freely. Again, location doesn't matter nearly as much as the fact that you are meeting in a safe environment where team members feel free to speak their minds.

Although the ScrumMaster will often act as and can be quite effective as the facilitator for the sprint retrospective, any capable team member can fulfill the role of retrospective facilitator. There are also times when bringing in a skilled, neutral, outside facilitator might be the best solution to help team members either get started doing retrospectives or to assist them through a particularly difficult or sensitive retrospective where a closely aligned, internal facilitator may be far less successful. Alternatively, in organizations with multiple Scrum teams with different ScrumMasters, it is often helpful and enlightening to all involved to have the ScrumMaster of one Scrum team facilitate the retrospective of a different Scrum team. We should establish who is going to facilitate the retrospective during the prework.

Approach

Figure 22.4 illustrates the sprint retrospective activity.

Inputs to the sprint retrospective include the agreed-upon focus for the retrospective and any exercises and materials that the team might decide to use during the retrospective. In addition, most retrospectives require at least some precollected, objective data. And one piece of input every attendee will bring without fail is her

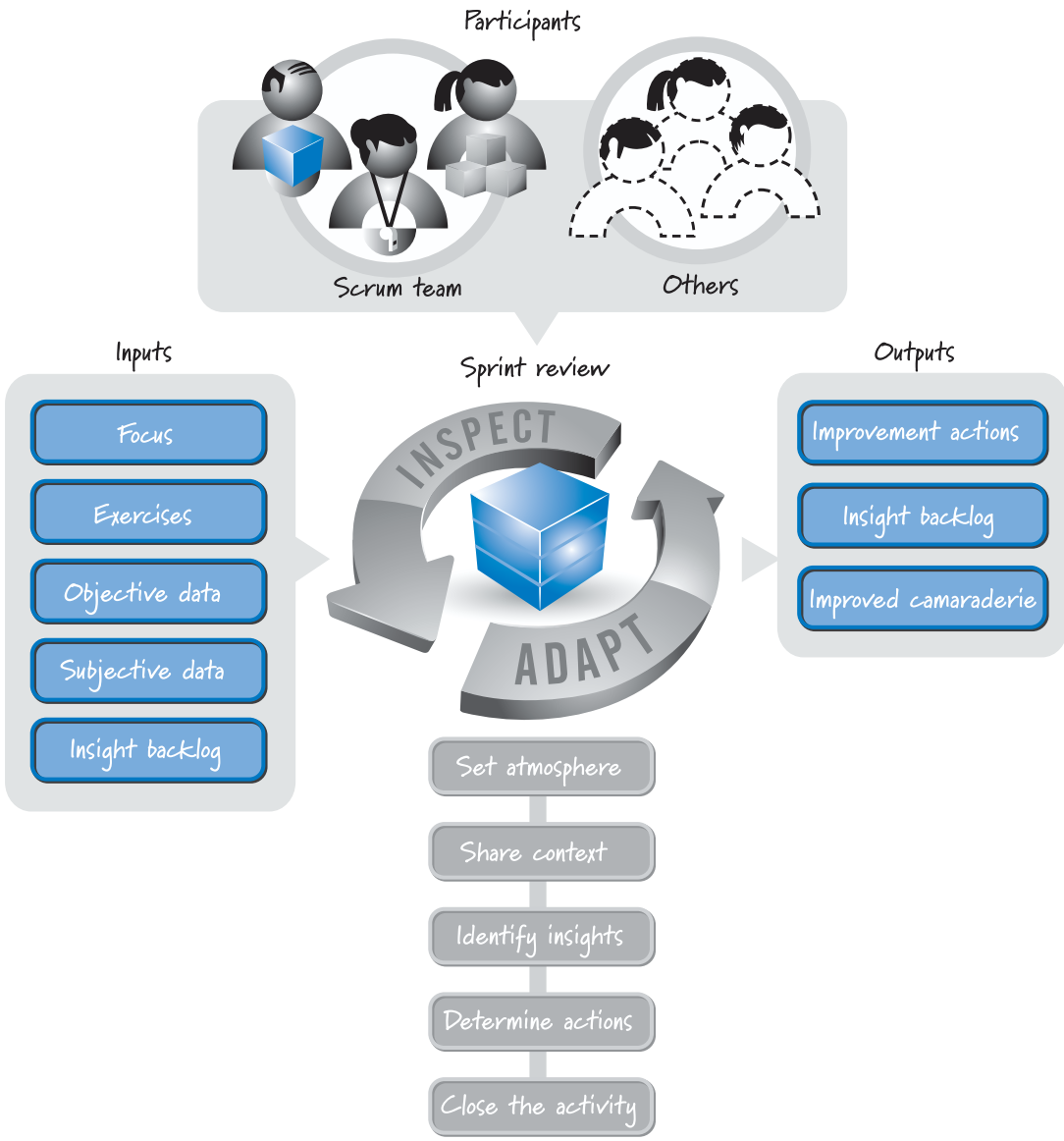


FIGURE 22.4 Sprint retrospective activity

own subjective data regarding the current sprint. Another retrospective input is a backlog of insights produced in previous retrospectives.

The outputs of the sprint retrospective include a set of concrete improvement actions that the team has agreed to perform in the next sprint. The outputs might also include a backlog of insights collected during the current retrospective that the team will not address in the upcoming sprint but might choose to address in the future. Team members should also expect improved camaraderie as an output from a retrospective.

While many retrospective approaches exist, most seek to answer the following questions:

- What worked well this sprint that we want to continue doing?
- What didn't work well this sprint that we should stop doing?
- What should we start doing or improve?

An approach (similar to one described by Derby and Larsen 2006) that I find useful is to set the atmosphere for the retrospective, create a shared context among the participants, identify insights that can lead to improvements, determine concrete improvement actions to take during the next sprint, and close the retrospective. These steps are shown in Figure 22.4 and explained in the paragraphs that follow.

Set the Atmosphere

During a retrospective, people are being asked to analyze the behavior and performance of their team and to make specific recommendations for how the team can improve itself. Putting the team (and by extension oneself) under a microscope can be an uncomfortable experience. So, a good way to start the retrospective is to establish an atmosphere that makes people feel comfortable participating.

People must feel it is safe to express their opinions without fear of retribution. Teams should have established ground rules, or a working agreement, which make it clear that expressing opinions and airing dirty laundry are safe things to do. It is helpful for the ground rules to make clear that the focus is on the organizational system and process, not the individuals, thus making it safe to explore what went wrong.

There will be times when problems are people problems; the retrospective is not the place to solve them. The retrospective is about improving the Scrum team's process, not about assigning blame or reprimanding individual behavior. When setting the atmosphere, ensure that the ground rules reinforce the concept of a blame-free environment.

It is also important to establish a precedent of active participation. We won't have a very effective retrospective if people assume a passive role. So, when setting the atmosphere, it's a good idea to get people talking just to prime the pump of participation. Some teams do something as simple as ask each participant to express in a few words her current feelings or energy level. It's not critical what question people are asked to answer, but that they are asked to say something to get in the mood of talking.

Share Context

A group of people can all experience the same event and yet interpret it quite differently. To successfully inspect the current sprint, it is important to get everyone on the same page so that they have a shared context.

To establish a shared context the participants must align their diverse individual perspectives (see the left side of Figure 22.5) into a shared team perspective (see the right side of Figure 22.5).

The left side of Figure 22.5 shows that each person might view the sprint differently based on her own experience during the sprint rather than have a more big-picture view of the sprint events, accomplishments, and shortcomings. If individual perspectives are allowed to dominate, the retrospective could degrade into a session of opinion debate rather than a session focused on actionable outcomes based on a shared context.

When establishing a shared context, therefore, it's imperative that you first ground the retrospective in an objective, big-picture view of the sprint. After getting everyone in a talking mood, share objective data, such as committed PBIs, PBIs completed, number of defects, and so on. (Exactly what specific objective data is relevant should be based on the retrospective focus.) While most of the objective data is typically gathered during the prework, some objective data can also be left for the participants to collect collaboratively during the retrospective. Doing this can help energize the team around the importance of that data. Whether done as part of the prework or as a group, gathering objective data is crucial for establishing a common foundation built on facts rather than opinions.

Just because we are grounded in objective data doesn't mean subjective data is irrelevant, however. Each person brings to the retrospective subjective data reflecting her interpretation of the sprint. If that subjective data is not exposed and discussed, participants might just assume that everyone else experienced the sprint in a similar

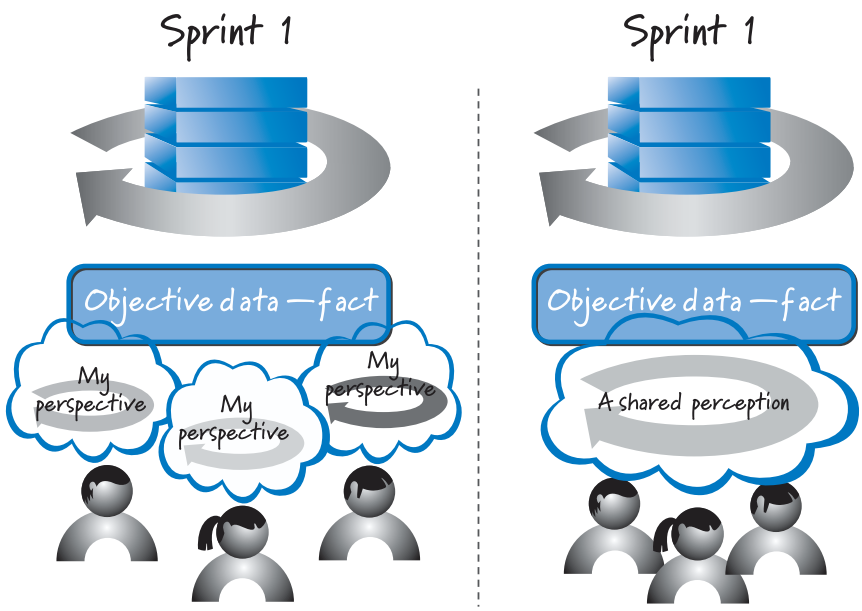


FIGURE 22.5 Aligning perspectives to create a shared context

way. This misalignment will make it difficult for people to understand one another's comments and suggestions.

There are a number of exercises that the participants can use to develop a shared context of both objective and subjective data. Two of the most common exercises are an event timeline and an emotions seismograph.

Event Timeline

Creating an **event timeline** is a simple yet powerful way to generate a shared artifact that visually represents the flow of events during a sprint. Events could include "Busted the build," or "Interrupted to fix production failure," or "Salina returned from holiday."

A common approach is to draw a timeline on a wall or whiteboard and have the participants put cards (or sticky notes) on the timeline representing meaningful events that occurred during the sprint (see Figure 22.6). Distributed teams could do the same exercise using an online-shared whiteboard.

The event cards are placed on the timeline in chronological order. This temporal view of events provides excellent visibility into the flow of activities during the sprint and also provides a context for quickly identifying missing or forgotten events.

To help visually categorize events, many teams use a variety of colored cards. Some do this to represent different event types (for example, green is a technical event, yellow is an organizational event, red is a personal event). Other teams use colors to represent feelings or energy levels (for example, green is a positive event, yellow is a neutral event, and pink or red is a negative event).

Emotions Seismograph

Many teams create an **emotions seismograph** as a complement to their event timeline. This is a graphical representation of the emotional ups and downs of the participants over the course of the sprint (see Figure 22.7). Creating an emotions seismograph helps expand the shared context beyond the objective data (what happened) to include some subjective data (how the team felt about it).

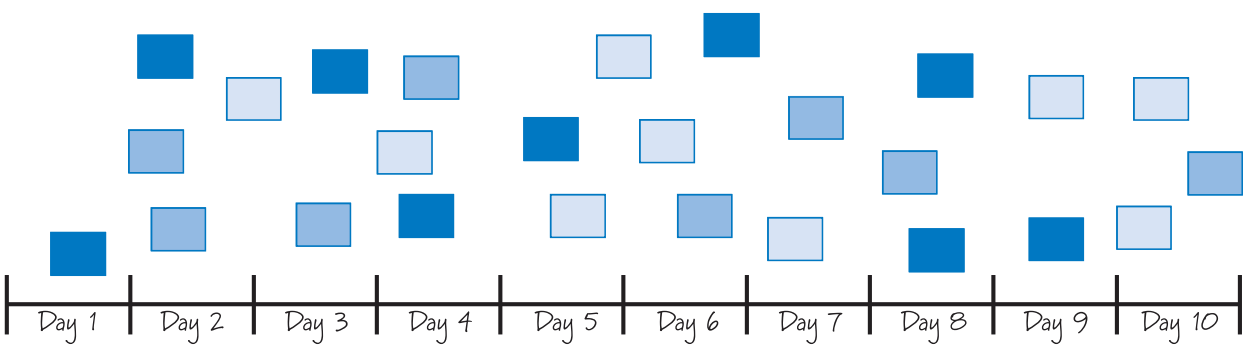


FIGURE 22.6 Sprint event timeline

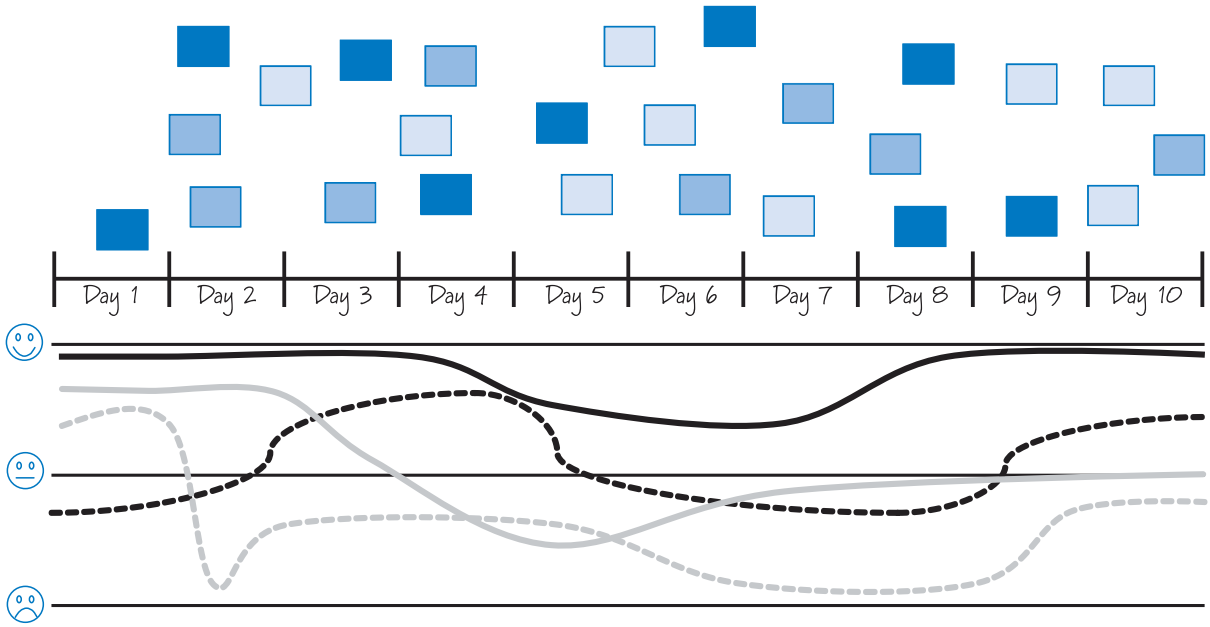


FIGURE 22.7 Emotions seismograph

To create the seismograph each participant is invited to draw a curve showing how she felt or what her energy level was like over the course of the sprint. It is frequently convenient to draw the seismograph directly under the event timeline so that the two sets of data can be visually correlated. Later, the participants can mine this data for interesting insights for process improvement.

Identify Insights

Once a shared context has been established, the participants can thoughtfully examine, understand, and interpret the data to identify process improvement insights. Doing this effectively requires a system-level (bigger-picture) focus. Focusing on just one aspect (having a more localized view) might cause teams to miss the bigger picture. A system-level focus also helps teams move past the superficial and identify the root causes of issues.

The participants should start by mining the shared context data. For example, they could look at their event timeline and emotions seismograph and ask the following questions to help uncover insights:

- What worked well?
- What didn't work well?
- Where are some opportunities to do things differently?

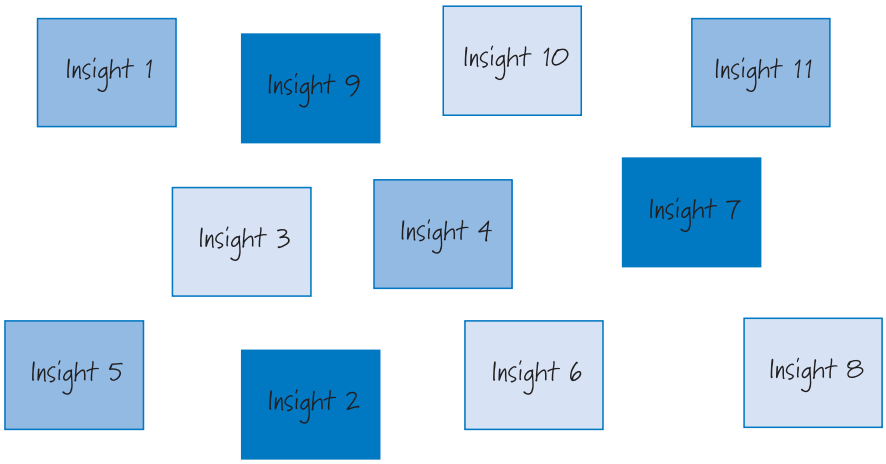


FIGURE 22.8 Retrospective insight card wall

Frequently participants are asked to brainstorm insights and then capture them on cards and place them on a shared wall or other surface so that everyone can see them (see Figure 22.8).

Another source for insights might be the team's **insight backlog**, a prioritized list of previously generated insights that have not yet been acted upon. If such a backlog exists, mine it to see which insights the participants would like to include and consider during the current retrospective. Any existing insights should be represented by cards and placed on the wall alongside the new insights.

Once the cards have been placed on the wall, the participants will need to organize them. To do this, many teams choose an exercise such as **silent grouping** to cluster the insights into meaningful groupings to indicate similar or duplicate cards (see Figure 22.9).

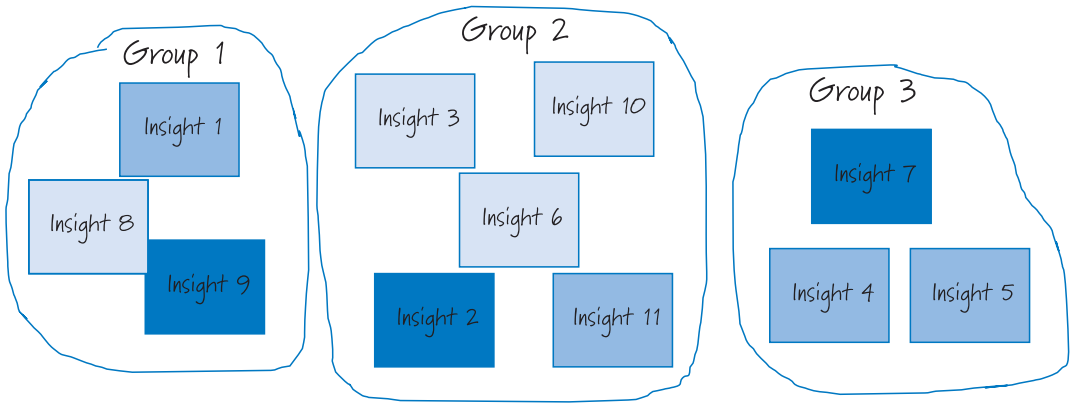


FIGURE 22.9 Insight cards clustered into similarity groups

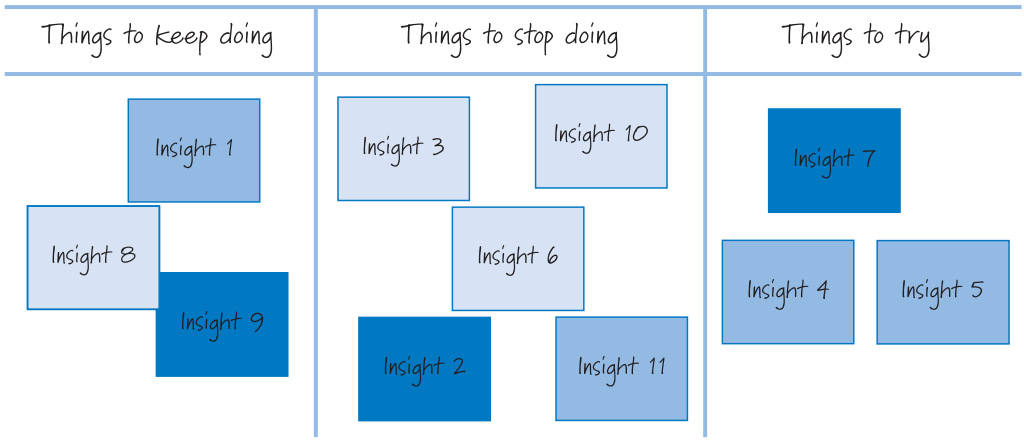


FIGURE 22.10 Insight cards placed into predetermined groups

During silent grouping, people collaboratively create the groupings without verbal discussion, relying only on the individual placement and movement of cards as a means of communicating and coordinating among the participants. Silent grouping is time efficient and effective.

Other teams prefer to divide the wall into category areas (such as things to keep doing, things to stop doing, things to try) before the retrospective begins. Then, as the insight cards are being created, the participants can place each card on the wall in the appropriate category (see Figure 22.10).

Even with preassigned categories, however, it is still effective and efficient to have the participants perform silent grouping to cluster similar cards within a category.

After creating a shared context and mining the data for insights, the participants should have identified many areas for improvement in their use of Scrum and, by extension, the way they work together to deliver value. Some of these insights might lead to deeper discussions among the participants to better understand underlying causes, or important patterns or relationships. When all the insights have been discussed and organized on the wall, it's time to determine what to do with all of the information.

Determine Actions

Insights are our ideas or perceptions of things that can be improved. To extract long-term value from these insights we need to move from discussing them to taking demonstrable actions to leverage them. For example, if the insight is "We're wasting too much time because the code management system keeps failing," the improvement action moving forward might be "Have Talya apply the vendor patches to the code management system to make it more stable." Talya, a member of the development team, can take this action in the next sprint.

The participants should also take time to review what happened to the improvement actions from the last retrospective. If those actions have not been completed (or even started), the participants need to know why before they start addressing new insights. They might choose to carry forward previous actions or prioritize them against the new insights they have just identified.

Selecting Insights

It is important to realize that retrospectives frequently identify many more improvement insights than the Scrum team and organization can digest and act on in a short period of time. So, the participants first need to determine which improvement insights to act on immediately and which can be deferred. Many teams have the participants prioritize the insights based on what they believe is most important or where they are most energetic about seeing improvement. Sometimes these two are not the same. We might agree that a particular improvement insight is important, but if there is no appetite to do the work required to leverage the insight, it might not be a good choice right now. If the participants are energetic about an insight, they are much more likely to take concrete actions to leverage it.

One popular way to prioritize insights is to use **dot voting**, as illustrated in Figure 22.11.

During dot voting, each participant is given a small number (perhaps three to five) colored dots. The participants then simultaneously place their dots on the improvement insight cards that they feel are the highest priorities to address. A person can put all of her dots on one card or spread them out over several cards. Once everyone has voted, the cards with the greatest number of votes should be considered first.

Exactly how many insights should the participants select to work on? Well, that depends on how much capacity the participants are able to dedicate to the insights and over what period of time.

Typically the period of time is the next sprint. So if the Scrum team is doing two-week sprints, it will likely consider insights that it can address over the next two-week

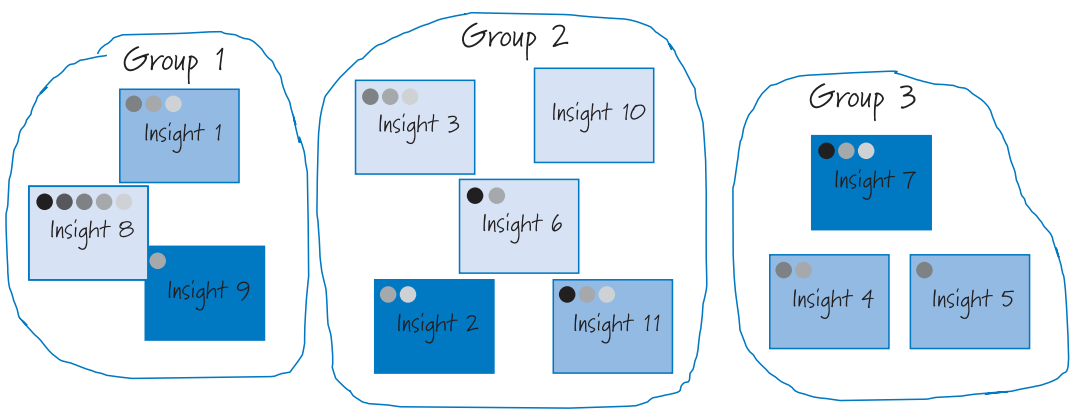


FIGURE 22.11 Example of dot voting

period. Even if an insight is too large to be fully addressed in the next sprint, the participants might choose to start working on it and make demonstrable progress against it.

The participants must also determine how much capacity they can dedicate to addressing insights during the next sprint (or whatever time period they are considering). If the team plans to dedicate time in the next sprint to actions from the previous retrospective, it will clearly affect the team's capacity for new actions identified in this retrospective.

Spending time working on insights, old and new, leaves less time for working on features. So, how much time should the team allocate today to address insights that can provide a larger payoff later? Answering that question really requires input from the product owner, which is one of the reasons it is important to have the product owner present at the retrospective. If the Scrum team doesn't specifically allocate time to work on improvement insights, a likely outcome is that insights won't get worked on.

Once we know the available capacity to work on insights, the participants can get a rough idea of which of the high-priority insights can be immediately addressed. However, the final decision can really be made only once the specific actions are determined.

Decide on Actions

At this point we have prioritized our insights and have some idea of the capacity we have to work on them. However, we get no measurable value from the retrospective until we define concrete, actionable steps to leverage our insights and improve the Scrum process.

Most actions will take the form of specific tasks that one or more Scrum team members will perform during the upcoming sprint. For example, if the insight is "It takes too long to determine when the code build breaks," the action might be "Have the build server send an email when the build is broken." This action requires task-level work on the part of one or more development team members. The team should determine who can do this work and how much time the work will take. Only then can the team be sure that working on a particular insight is doable within the available capacity.

Not all insights require specific task-level work. For example, an insight like "Be respectful to one another and show up to the daily scrum on time" should require little (if any) task-level work by the team. While there is a real action, "People should actually make the effort to show up on time," it will not reduce the team's capacity.

Sometimes the actions represent impediments that the ScrumMaster might own but someone else in the organization has to resolve. For example, the insight might be "We can't get PBIs done because we need the latest version of a third-party vendor's software to test against." The action might be "Nina will work with our procurement department to obtain the latest vendor update." So Nina, the ScrumMaster, will work with the procurement people to address the third-party vendor issue that is impeding

the team from getting PBIs done. This action will start in the next sprint, will need some capacity from the ScrumMaster, and may require several sprints to be resolved.

When determining the proper actions, we need to remember that it might not be possible to immediately address the insight. Instead, we might need to *explore* the insight before we can actually make an improvement. In such cases the proper action might be to investigate and collect data during the next sprint so that we can better understand the problem.

For example, the insight might be "We're puzzled by why two components that are fully tested and have their own automated test suites fail when they are combined into a cross-component automated test suite where each component is still individually executed." At this point there isn't a specific action the Scrum team members can take to address this insight because they really don't understand what is going wrong. However, the team can create an action for specific team members to explore this issue in the next sprint, and the team can determine how much capacity to allocate for exploration.

Insight Backlog

As I mentioned earlier, many teams create an *insight backlog* (sometimes called an *improvement backlog*) to hold any issues that are identified during a retrospective but cannot be worked on immediately. The idea is that at the next sprint retrospective the participants can choose to use the insights in the backlog as candidates to be prioritized against new insights when determining where to focus time in the next sprint. Of course, the insight backlog should be groomed periodically to ensure that its contents remain valuable insights.

Other teams simply discard any insights that they choose not to work on in the next sprint. The thinking is that if an insight is truly important, it will be identified again at the next retrospective.

Close the Retrospective

Once the final improvement actions have been determined, the participants close out the retrospective. Many close by recapping what actions the team has decided to take based on what the participants learned. This might be as simple as describing each committed action item and who is going to work on it.

Closing is also a good time to appreciate people and their participation. Each participant should say a few kind words of appreciation regarding the contributions made by others. Be sure to also recognize any non–Scrum team members who took time out of their busy schedules to participate in the retrospective.

Finally, it's a good idea to spend a few minutes asking the team for suggestions to improve the team's approach to performing a retrospective. A retrospective is, after all, part of the Scrum framework and as such should also be subject to inspection and adaptation.

Follow Through

To ensure that what happens in the sprint retrospective does *not* just stay in the sprint retrospective, the participants should follow up on the actions they chose to complete. Some actions (such as that everyone shows up on time for daily meetings) need only to be reiterated and reinforced by the team members and the ScrumMaster. Others will need to be addressed during the forthcoming sprint-planning activity.

Frequently the easiest way to handle the improvement actions is to populate the sprint backlog with tasks corresponding to each action prior to bringing in new features. The team's available capacity to work on new features would then be adjusted downward by the estimated time these improvement tasks will take. Honestly, any approach that allows the team to make a good commitment at sprint planning while at the same time affording it the opportunity to work on the improvement actions is a good approach.

One approach that does *not* work is to have an "improvement plan" for the team that is separate from the work it will do each sprint. This two-pronged approach will almost always lead to the improvement plan being subordinate to the typical feature-driven sprint plan. To ensure that the improvement actions do take place, don't separate; integrate!

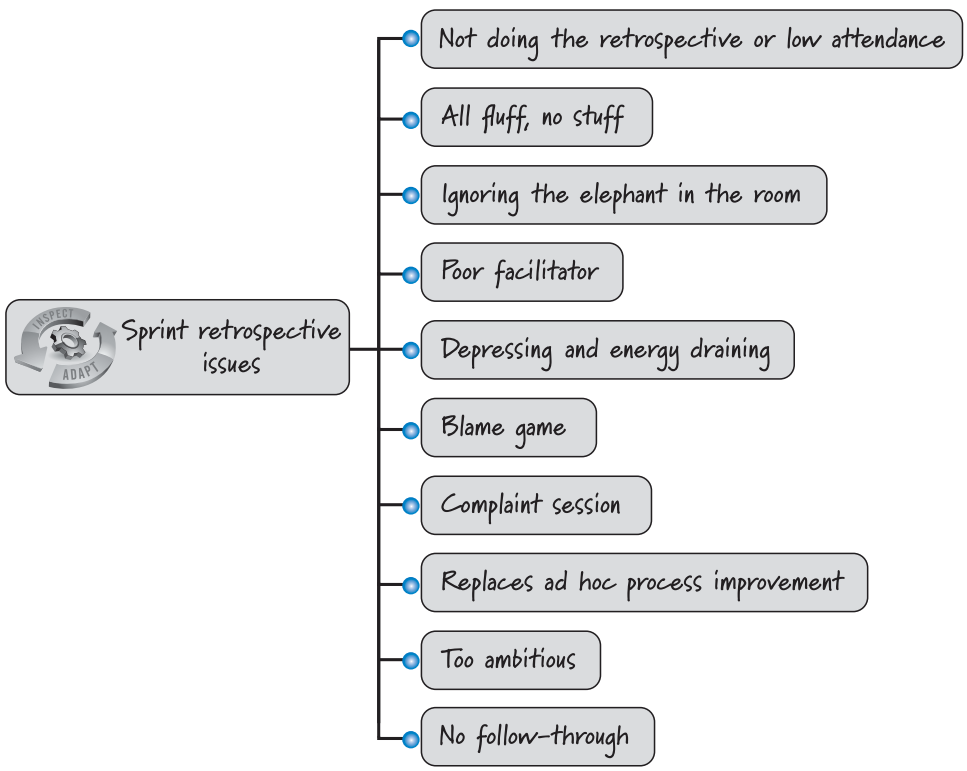


FIGURE 22.12 Sprint retrospective issues

Actions that do not require team member time will likely find a home on the ScrumMaster's impediment list. And actions that are destined for other teams or the organization as a whole can be placed into the appropriate backlog for the people who are expected to do the work; the ScrumMaster typically follows up with the external parties to help ensure that these actions actually get done.

Sprint Retrospective Issues

Sprint retrospectives are not without issues. Having worked with many organizations using Scrum, I have noticed a number of common issues (see Figure 22.12).

An unfortunate issue is when teams simply don't do the sprint retrospective, or when they do, attendance is low. The reasons for both tend to be similar. If people are assigned to multiple teams, scheduling conflicts could prevent them from attending. This is an organizational dysfunction that managers need to address. Or perhaps team members are just bored or disengaged, or they have not really bought into using Scrum. Others might think that doing anything other than their particular work isn't worth their time (for example, they believe that anything other than coding or testing is just wasteful). Often these issues stem from naiveté regarding Scrum and its focus on continuous improvement. Other times it's just the opposite—team members believe they have reached the pinnacle of Scrum usage and therefore have nothing further to learn from the sprint they just performed, from their teammates, or from their own success or failure. If people don't see the value of doing a retrospective or of their attendance, consider dedicating some of or the entire next retrospective meeting to exploring this value issue.

Sometimes low attendance happens because it is inconvenient for remote participants to join by phone or video conferencing. If remote participants find attending the retrospective inconvenient because of when it is scheduled, consider changing or rotating the time so that no single location is always inconvenienced. If it is inconvenient because it is just hard to participate remotely, reconsider the current telecom infrastructure and how the exercises are being conducted to better incorporate remote participants.

Some retrospectives are very busy but really don't achieve anything actionable. I call these the *all fluff, no stuff* retrospectives. If all we get is fluff, we're wasting our time. Consider bringing in an outside, experienced retrospective facilitator to help the participants get to the real stuff.

Other retrospectives are fascinating to observe. There is clearly a critical issue that is having a dramatic effect on the team, but nobody will even bring it up. To reuse the old adage, the participants are ignoring the elephant in the room. There is probably a safety issue that is preventing people from discussing the elephant. The ScrumMaster should take a leadership role in helping the team and organization address the safety impediment first.

Other times the retrospective is just poorly facilitated. The facilitator, perhaps a new ScrumMaster, is trying her best but it's clearly not working. Perhaps an outside facilitator should be used for a few retrospectives.

Some retrospectives are downright depressing and energy draining. Perhaps the sprints are not going well and people view the retrospective as an activity that just compounds the misery by making them relive it. Consider spending a bit more time to set the appropriate atmosphere at the start of the retrospective. Also, an outside facilitator might be more effective at helping people stay focused on positive improvements.

Frequently retrospectives are depressing because people start playing the blame game and finger-pointing. The facilitator must extinguish this behavior as soon as it occurs to prevent a cascade of finger-pointing.

Other times a retrospective can degrade into a complaint session. Perhaps some people view it as therapeutic to just come and complain about the way things are (or at least how they perceive things to be). They have no desire to improve, just the desire to complain. Consider inviting people to the retrospective who can actually effect real change. Then have a face-to-face dialogue with them instead of complaining in their absence.

Another unfortunate situation is where participants consider the retrospective to be *the* time to do process improvement, thereby diminishing ad hoc process improvement during the sprints. A retrospective is a great time for the team to reflect on a period of work and discuss how to make things better, but it was never intended to be the replacement for ad hoc process improvement. The ScrumMaster should proactively promote healthy ad hoc process improvements throughout the sprint.

Sometimes our desires are bigger than our abilities. New teams that are energized and focused on really getting better can frequently become overly ambitious and set improvement goals that are totally unrealistic. Doing so will only lead to a big letdown when the team fails to meet its ambitious goals. The ScrumMaster should be vigilant and remind the participants of their available capacity to do improvements and help them moderate their ambitions.

Perhaps the biggest issue of them all is when there is no follow-through to actually work on the improvement actions identified during the retrospective. If we're not going to follow through, there's no need to waste our time on retrospectives. The ScrumMaster has a leadership role in helping the team constantly improve its process. If there is no follow-through, the ScrumMaster needs to be aggressive about working with the team to identify the root cause and helping team members address the impediment.

Closing

Sprint retrospectives are a time for the team to reflect on how well it is using Scrum and to propose improvements. The retrospective is a collaborative activity among the Scrum team members (and any non–Scrum team members on an as-needed basis).

Once the retrospective prework is completed, the basic flow of the retrospective is to set the atmosphere to have a successful retrospective, get everyone on the same page by creating a shared context grounded in data, identify improvement insights, determine improvement actions, and then close the retrospective. After closing the retrospective, it is critical that the participants follow up and carry out the improvement actions so that the team is more effective during the next sprint. It is also important to keep a watchful eye out for issues that might prevent the retrospective from being successful and to quickly act on them.

Chapter 23
The Path Forward

In the first 22 chapters of this book, I have laid out the Scrum framework and explained what I believe to be essential Scrum. You should now understand the mechanics of using Scrum to deliver innovative solutions. You should also have a good sense of why Scrum prescribes particular roles, practices, artifacts, and rules. Now you are ready to define your path forward. In this chapter I discuss the idea that there is no universal, final target for your Scrum implementation; instead, you need to define your own unique route towards agility. I end by describing the role of best practices and how to use Scrum, with its iterative and incremental approach, as the basis for discovering your own path forward.

There Is No End State

Every organization has a vision for what it wants to achieve. Using Scrum can help organizations manage the work to achieve that vision. Being highly proficient with Scrum and thus being more agile, however, is not the end goal, but rather a means of more effectively and economically achieving business goals. So, how do you know when you're done with Scrum?

The thing is, there is no definition of done for a Scrum adoption or transition effort. There is no agile maturity model like CMMI (SEI 2010), where the goal is to try to reach level 5. Trying to define "done" for your Scrum implementation presumes that when you achieve that state, you can't get any better. This flies in the face of Scrum being a form of continuous improvement, where you are always working to better align your use of Scrum to the complex world in which you must develop products.

Worse yet, if we tried to define such an end state for the industry, it would further assume that the final target state would apply to every organization, even those that develop radically different types of products under very different circumstances.

To say, "I have finally achieved agility!" is a meaningless comment. As Mike Cohn nicely summarized, "Agile is not something you become, it's something you become more of" (Cohn 2010). There is no end state that you can call agile or Scrum. Instead, becoming more proficient with Scrum and more agile is a process of continuous, never-ending improvement aimed at improving your bottom line.

Discover Your Own Path

Just as no one can tell you where your Scrum implementation should end, no one can lead you down a predefined path that will guarantee success. Instead, you must learn, inspect, and adapt your way forward based on your organization's own unique goals and culture and the ever-changing complex environment in which you must operate. Trying to follow someone else's path and leveraging their learning might feel like the fast track to becoming agile. However, no two organizations, or even teams within an organization, are the same. Following someone else's path might well lead you to precisely the wrong location.

You can't circumvent your own learning process. Instead, you need to quickly close your own learning loops and inspect and adapt based on what you learn. I'm not suggesting that you ignore those who have preceded you down the agile path. Examine what they have done and how it worked for them, but discover your own path for becoming more agile.

Sharing Best Practices

If we aren't supposed to follow others' paths, how do best practices fit into the picture? Just as there is no one path to follow, there is no one set of best practices across all organizations.

When I'm asked to describe "best practices" that other organizations are using to become more agile, I give examples. However, I always provide the relevant context of the other organizations so that the people who are asking can evaluate whether it would be sensible to adopt a similar approach in their organization. Even when scaling up within a single organization, we need to be cautious about universally applying best practices. Many organizations try to describe what a successful Scrum team is doing, capture it, and then institutionalize it as a best practice. Doing so can be harmful, because these are individual team approaches that may not work for other teams.

You might notice I just used the word *approach* several times when referring to best *practices*. Let's talk for a moment about that distinction. Throughout this book I have used the term *practice* to mean a core or essential aspect of Scrum. An approach is a particular implementation of a Scrum practice. When people ask me about best *practices*, I take that to mean best *approaches*.

While two different teams or organizations have unique Scrum implementations, each should adhere to the same Scrum practices. Both, for example, should have Scrum teams made up of a product owner, ScrumMaster, and development team. Both should perform sprint planning, daily scrums, sprint reviews, and sprint retrospectives. However, I expect that each team (or organization) will have its own unique approaches to performing these practices. Let me illustrate by example.

The daily scrum is a core Scrum practice. If you don't do it, you aren't doing Scrum. During the daily scrum each team member updates and synchronizes with all other team members to get a shared picture of the full scope of work. But when the daily scrum starts, which person should provide his update first? Scrum doesn't define this. Each team will have its own approach.

For example, while working with a team in Vancouver, Canada, I learned of an interesting approach to deciding who speaks first. On this team, at the start of each daily scrum, the ScrumMaster would toss a toy stuffed moose into the air. Whoever caught the moose would speak first, and then the rest of the team members would speak in turn, moving to the left of the person who caught the moose. This simple, kind-of-silly-but-fun approach worked very well for the team in Vancouver.

As it turns out, the Vancouver team had a sister team in China that was formed some months after the Vancouver team had been established. A member of the China team asked the Vancouver team for a "policy or best practice" for determining who should speak first at the daily scrum. The Vancouver team told the person in China that in Vancouver they "tossed the moose" each daily scrum to figure that out. Apparently, tossing the moose took on a whole different meaning when translated into Chinese! An approach that worked well for the Vancouver team would not work at all for the China team. The China team adopted its own approach, as well it should.

Scrum defines core practices that must be followed. It is left to each team, however, to determine the approaches (or best practices) that work best. Approaches, therefore, are unique to each team, and they can and should be reused by other teams only if they make sense in the context of those other teams.

Using Scrum to Discover the Path Forward

Whether you are new to Scrum or are already using Scrum to develop products, you can use the principles of Scrum to help guide you on the path forward. Mike Cohn, in *Succeeding with Agile* (Cohn 2009), goes into great detail describing this approach, and I refer you to his excellent book for a detailed treatment of the topic.

I describe the essence of this approach with an example. In 2007 I was retained to train and coach a large, multinational organization on its adoption and use of Scrum. The organization had 100 IT members in New York City and 400 IT members in Mumbai, India. At any one time the IT organization had about 45 development efforts in flight.

The organization decided that any new team that was going to use Scrum should have a coach available to help it. With limited initial coaching capacity, it was unreasonable to transition the entire IT organization over to Scrum at once. So, as is typical in such environments, the organization selected a small number of pilot efforts to focus on first. The goal was to incrementally move other teams over to Scrum as the organization's Scrum coaching capacity increased via the on-the-job training of internal coaches.

These pilot efforts crossed the spectrum from simple systems maintenance to larger, new-product development. Based on this diversity, each Scrum team implemented its own version of the Scrum framework, using approaches aligned with the people on the team and the work they had to perform. A wiki was used to "syndicate" the approaches used by each team to assist with overall organizational learning and share what was working for other teams.

Several months into the adoption, it was time to scale the use of Scrum from the team level to the organizational level. At that point we created what Cohn calls an ETC or Enterprise Transition Community (Cohn 2009). In our case the ETC was called the Working Software Group. That group of managers and executives maintained a backlog of improvement-related items and ran three-week sprints against that backlog. The items in that backlog represented organizational change initiatives (such as "Update the compensation model to be more team focused") or significant impediments that were blocking one or more Scrum teams ("Improve server stability so that teams can complete their testing").

By working in sprints against this improvement backlog, the organization was able to iteratively and incrementally make progress down its own path to successfully adopting Scrum. There was no predetermined end state for the organization's use of Scrum. Trying to create one up front would have been as wasteful as trying to create a full, complete requirements specification for a totally new, never-been-built product that no one understood well.

Instead, the Working Software Group took input from the Scrum teams and stakeholders and made incremental improvements to the organizational structure to better align itself with agile values. Through continuous learning, inspecting, and adapting, the organization determined an appropriate path forward that was aligned with the overall organizational business goals.

This ETC-type pattern is quite common today. Many organizations realize that using Scrum to adopt Scrum is a sensible approach to iteratively and incrementally becoming more agile.

Get Going!

I am frequently amused when the same people who believe that it is not possible to get the requirements for a product correct up front, and therefore want to use Scrum for development, will in the next breath explain how they aren't quite ready to start using Scrum because they haven't worked out all of the details of their Scrum approach! This type of thinking is poorly aligned with fundamental Scrum principles.

When employing Scrum, you shouldn't worry about getting things perfect up front. You can't! And trying to be perfect up front will force you to guess at the expense of important learning that you will get only by applying Scrum and seeing what happens. In my experience, most teams' first couple of sprints aren't all that

pretty. That's OK. My only expectation of any Scrum team is that it be better in the next sprint than it was in the previous sprint. So, don't delay getting started. Whatever you think you know now about your use of Scrum, imagine how much more you will really know after you start and finish the next sprint!

Also, don't expect your Scrum adoption to be problem free. I can guarantee that at some point your organization will encounter impediments that make performing Scrum difficult. Scrum makes visible the dysfunctions and waste that prevent organizations from reaching their true potential. What it does not do is tell organizations how to solve those issues. That hard work is up to the people in the organization.

The status quo is a powerful force. It is frequently easier for people to ignore Scrum or change Scrum than it is to change long-held organizational processes, rules, or behaviors. And a culture that is outright hostile toward being shown its dysfunctions will quickly extinguish the bright light that is exposing what lurks in the shadows. To counteract this tendency, be a steadfast and patient force for change in your organization. Understand that resistance to change is natural. Help overcome the worst of it by educating others about the principles that underlie Scrum and the goals you are trying to achieve. Work with them, rather than against them, to chip away at the obstacles that prevent your team, your product development effort, and your organization from realizing the full benefits of your Scrum implementation.

My hope is that this book has provided you with the essential Scrum knowledge to shine a bright light that will illuminate your path forward. I wish you the best of success on your journey with Scrum.

This page intentionally left blank

GLOSSARY

Overview

Entries in this glossary are arranged alphabetically. An entry may be a single word, such as *Scrum*, a phrase, such as *acceptance criteria*, or an acronym, such as *TDD*. If a term has more than one definition, the definitions are numbered.

The following cross-references are used to show a term's relationship to other terms in the glossary:

- *See* refers to a preferred term or to a term whose definition serves to define the term in question.
- *See also* refers to a related term.
- *Synonymous with* refers to a synonym or term with a nearly identical meaning.
- *Contrast with* refers to a term with a substantially different meaning.

Definitions

A

acceptance criteria. 1. The external quality characteristics specified by the product owner from a business or stakeholder perspective. Acceptance criteria define desired behavior and are used to determine whether a product backlog item has been successfully developed. 2. The exit criteria that a component or a system must satisfy in order to be accepted by a user, customer, or other authorized entity (IEEE 610).

acceptance test. 1. The testing carried out to verify that the acceptance criteria have been met. 2. A test that defines the business value each product backlog item must deliver. It may verify functional requirements or nonfunctional requirements such as performance or reliability. It is used to help guide development (Crispin and Gregory 2009). 3. Formal testing with respect to user needs, requirements, and business processes conducted to determine whether or not a system satisfies the acceptance criteria and to enable the user, customers, or other authorized entity to determine whether or not to accept the system (IEEE 610).

acceptance-test-driven development (ATTD). A technique in which the participants collaboratively discuss acceptance criteria, using examples, and then distill them into a set of concrete acceptance tests before development begins. Synonymous with *specification by example*.

accuracy. How close an estimate is to the actual value—the proximity of the measure to its true value. For example, estimating that the product will ship in October 2015 is accurate if the product ships any day during October 2015. Contrast with *precision*.

activity. 1. A Scrum practice that involves taking action or performing a process, for example, sprint-planning activity, daily scrum activity, sprint review activity, and sprint retrospective activity. 2. In a general sense, the work performed by the Scrum team members such as writing code, performing tests, creating estimates, and so on. See also *practice*.

adaptation. One of the three pillars of empirical process control; feedback is used to make an adjustment to the work product being developed or the process by which it is being developed. See also *empirical process control, inspection, transparency.*

agile. 1. A specific set of values and principles, as expressed in the Agile Manifesto (Beck et al. 2001). 2. An umbrella term used for a group of related approaches to software development based on iterative and incremental development. Scrum is an agile approach to development. See also *Extreme Programming, Kanban, Scrum*.

all-at-once product development. Doing all types of work (for example, analysis, design, coding, integrating, and testing) opportunistically within a single iteration.

all-before-any. A characteristic of a sequential development process, where the work product from a previous step in a process is transferred to the next step using a batch size of 100%. See also *batch size*.

anticipatory process. See *plan-driven process.*

approach. A specific way to realize a practice or activity. For example, Scrum specifies a sprint retrospective. How a team chooses to perform a sprint retrospective is its approach, which may be different from the approaches of other teams. See also *activity, practice*.

artifact. A tangible by-product produced during product development. The product backlog, sprint backlog, and potentially shippable product increment are examples of Scrum artifacts. See also *practice*.

assumption. A guess, or belief, that is presumed to be true, real, or certain even though there is no validated learning to know that it is true. Contrast with *validated learning.*

ATTD. See *acceptance-test-driven development*.

B

batch size. The cardinality of a set of items to be processed at some future step. See also *work in process*.

Boy Scout rule. 1. Always leave the campground cleaner than you found it. If you find a mess on the ground, clean it up regardless of who might have made the mess. 2. Every time you are in an area of the code doing work, always leave the code a little cleaner, not a little messier, than you found it. See also *technical debt*.

burndown chart. A graph that shows on the vertical axis the *quantity of work* (in either hours or product backlog item units) remaining over *time*, which is shown on the horizontal axis. Because less and less work should remain over time, the general trend in the graph is to burn down to a point where no work remains. We can show projected outcomes on burndown charts by calculating a trend line to see when work might be completed. Contrast with *burnup chart*.

burnup chart. A graph that shows the progress of work toward a *goal line* associated with a value on the vertical axis. As work is completed over *time* (the horizontal axis), the progress line moves up (burns up) to be nearer to the goal line. We can show projected outcomes on burnup charts by calculating a trend line to see when work might be completed. Contrast with *burndown chart*.

C

cadence. A regular, predictable rhythm or heartbeat. Sprints of consistent duration establish a cadence for a development effort. See also *synchronization*.

capacity. 1. The quantity of resources available to perform useful work. 2. A concept used to help establish a WIP limit by ensuring that we only start work to match the available capacity to complete work. See also *work in process*.

ceremony. A ritualistic or symbolic activity that is performed on well-defined occasions. Some people refer to the core Scrum activities of sprint planning, daily scrum, sprint review, and sprint retrospective as ceremonies. See also *activity, unnecessary formality*.

chaotic domain. 1. A situation that requires a rapid response. We are in a crisis and need to act immediately to prevent further harm and reestablish at least some order. 2. One of the domains in the Cynefin framework. See also *Cynefin*. Contrast with *complex domain, complicated domain, disorder domain, simple domain*.

chickens. A metaphor used by some Scrum teams to indicate that people are invested in the goal of the Scrum team, but at a level of involvement (not accountable) rather than commitment. Best used to refer to people outside of the Scrum team. Derived from an old joke about a chicken and a pig: "In a ham-and-eggs breakfast, the chicken is involved, but the pig is committed." Contrast with *pigs*.

chief product owner. The overall product owner within a product owner team on a large development effort. See also *product owner.*

commitment. The act of binding oneself to a course of action. Scrum encourages commitment. Commitment means that during both good times and bad, each team member is dedicated to meeting the team's collective goal. Contrast with *forecast.*

complex adaptive system. A system with many entities interacting with each other in various ways, where these interactions are governed by simple, localized rules operating in a context of constant feedback. Examples include the stock market, the brain, ant colonies, and Scrum teams.

complex domain. 1. A situation in which things are more unpredictable than they are predictable. If there is a right answer, we will know it only with hindsight. 2. One of the domains in the Cynefin framework. See also *Cynefin.* Contrast with *chaotic domain, complicated domain, disorder domain, simple domain.*

complicated domain. 1. A situation in which there might be multiple right answers but expert diagnosis is required to figure them out. 2. One of the domains in the Cynefin framework. See also *Cynefin.* Contrast with *chaotic domain, complex domain, disorder domain, simple domain.*

component team. A team that focuses on the creation of one or more components of a larger product that a customer would purchase. Component teams create assets or components that are then reused by other teams to assemble customer-valuable solutions. Contrast with *feature team.*

conditions of satisfaction. The conditions under which a product owner would be satisfied that a product backlog item is done. Conditions of satisfaction are acceptance criteria that clarify the desired behavior. See also *acceptance criteria.*

confidence threshold. 1. The definition of done for envisioning (product-level planning). 2. The set of information that decision makers need in order to have sufficient confidence to make a go/no-go funding decision for more detailed development.

continuous delivery. See *continuous deployment.*

continuous deployment. Deploying each new feature to users immediately after it is built, integrated, and tested. Synonymous with *continuous delivery, integration.*

continuous integration. A technical practice where members of a single team or multiple teams integrate their work as frequently as is practical. See also *integration, technical practices.*

cost of delay. The financial cost associated with delaying work or delaying achievement of a milestone. Cost of delay emphasizes the concept that time has a real financial cost, and to make economically sensible trade-offs it is important to know that cost.

cross-functional team. A team composed of members with all the functional skills (such as UI designers, developers, testers) and specialties necessary to complete work that requires more than a single discipline.

customer uncertainty. Uncertainty surrounding who the customers of a product are. See also *uncertainty*. Contrast with *end uncertainty, means uncertainty.*

Cynefin. A sense-making framework that helps us understand the situation in which we have to operate and decide on a situation-appropriate approach (Snowden and Boone 2007).

D

daily scrum. A synchronization, inspection, and adaptive planning activity that a development team performs each day. This core practice in the Scrum framework is timeboxed to no more than 15 minutes. Synonymous with *daily stand-up*. See also *inspect and adapt*.

daily stand-up. A common approach to performing a daily scrum whereby the participants stand for the entirety of the activity. Standing up promotes brevity and helps ensure that the activity does not exceed its timebox. See *daily scrum*.

DEEP. An acronym coined by Roman Pichler and Mike Cohn for remembering a set of criteria used to evaluate the quality of a product backlog. The criteria are *Detailed appropriately, Emergent*, *Estimated*, and *Prioritized*. See also *product backlog.*

defined process. A process with a well-defined set of steps. Given the same inputs, a defined process should produce the same output every time (within a defined variance range). Contrast with *empirical process control*.

definition of done. 1. A checklist of the types of work that the team is expected to successfully complete by the end of the sprint, before it can declare its work to be potentially shippable. A bare-minimum definition of done should yield a complete slice of product functionality, one that has been designed, built, integrated, tested, and documented and will deliver validated customer value. 2. Sometimes described as the acceptance criteria that apply to all product backlog items. Contrast with *definition of ready.*

definition of ready. A checklist of conditions that must be true before a product backlog item is considered ready to pull into a sprint during sprint planning. Contrast with *definition of done*.

development team. A self-organizing, cross-functional team of people who collectively are responsible for all of the work necessary to produce working, validated assets. One of the three roles that constitute every Scrum team. See also *cross-functional team, product owner, ScrumMaster, Scrum team*.